

Real-Time IoT Pipeline

1. Pipeline Overview

The pipeline ingests IoT telemetry events from battery management systems (BMS) and IoT device, processes them, and writes to two Delta Lake layers:

- Bronze layer - raw append-only historical events (~300M records/day)
- Silver layer - two upserted tables for latest device state and valid sensor readings

2. Data Volume & Throughput

Metric	Value
Daily record volume (Bronze)	~300 million records/day
Input rate (Bronze stream)	~4,157 rec/s
Processing rate (Bronze stream)	~4,137 rec/s
Input rate (Silver stream)	~16,600 rec/s
Processing rate (Silver stream)	~15,900 rec/s
Trigger interval (both streams)	30 seconds
Batch duration (Bronze avg/latest)	1.3 min / 1 min
Batch duration (Silver avg/latest)	1.7 min / 2.1 min
Kafka topic	normalized-iot-events
Kafka brokers	AWS MSK (ap-south-1), 3 brokers

3. Current Databricks Infrastructure

3.1 Cluster Configuration

Parameter	Value
Job name	iot_events_prod
Spark version	14.3.x-scala2.12 (DBR)
Driver node	r5d.4xlarge (1 node)
Worker nodes	r5d.4xlarge, autoscale min 1 / max 1
Node spec	16 vCPU, 128 GB RAM, 600 GB NVMe SSD
Availability	SPOT_WITH_FALLBACK
Availability Zone	ap-south-1a
Execution mode	Single User (spark.master: local[*])
Shuffle partitions	200
Parquet compression	ZSTD level 9
Job type	Continuous (no schedule, always-on)

3.2 Key Spark Configurations

Config Key	Value	Purpose
spark.sql.shuffle.partitions	200	Tuned for single-node throughput
spark.databricks.delta.autoCompact	auto	Auto file compaction
spark.databricks.delta.optimizeWrite	true	Optimal file sizing on write
spark.sql.streaming.stateStore.compression.codec	zstd	State store compression
spark.sql.streaming.minBatchesToRetain	2	Minimal checkpoint retention
spark.sql.streaming.stateStore.maintenanceInterval	2min	State cleanup interval
spark.sql.streaming.stateStore.timeout	10min	State expiry timeout
spark.cleaner.ttl	3600	Metadata cleanup (1 hour)
delta.feature.liquidClustering.enabled	true	Delta liquid clustering
parquet.compression.codec.zstd.level	9	Parquet ZSTD compression level

4. Pipeline Architecture

4.1 Source

- Kafka topic: normalized-iot-events on AWS MSK (3-broker cluster, ap-south-1)
- Message format: Apache Avro, schema loaded from schemav1.avsc
- Deserialization: from_avro() with PERMISSIVE mode (malformed messages skipped)
- Kafka group ID: databricks_dev_wrk
- Starting offsets: latest (not replaying historical data)

4.2 Stream 1 - Bronze IoT Append (process_iot)

Writes raw parsed events to the Bronze Delta table, partitioned by year/month/day.

Step	Description
Parse Avro	Decode Kafka value bytes using Avro schema
Latency columns	Compute timeLatency (pushedAt - ts) and consumedLatency (consumedAt - pushedAt) in minutes
Timestamp conversion	Convert ts (epoch ms) to IST timestamp
Partition columns	Add year, month, day, insert_date from ts
Alarm field validation	Check alarms struct fields against expected rename_map; raise error on unexpected fields
Alarm field cast	Cast all alarm struct sub-fields to string using rename_map aliases
Write	Delta append, partitioned by year/month/day, mergeSchema=true
Target table	{catalog}.bronze.iot

4.3 Stream 2 - Silver Upsert (upsert_to_delta)

Maintains two Silver Delta tables with the latest device state per deviceID.

Step	Description
Latest row selection	Window by deviceID ordered by ts DESC, pick row_num = 1
BMS validity check	isBMSValid = 1 if 0 < voltage < 70
IoT validity check	isIoTValid = 1 if lon > 0
BMS latest state	For valid BMS rows: latest by ts, columns bmsLastTs, voltage, soc, current, temperature, cellVolt, cellTemp, alarms, bmsLastLat, bmsLastLon
IoT latest state	For valid IoT rows: latest by ts, columns iotLastTs, lat, lon
Join	latest (deviceID, ts) LEFT JOIN bms_valid LEFT JOIN iot_valid on deviceID
Upsert - iot_events_latest_valid	MERGE on deviceID; update BMS fields only if source.bmsLastTs is newer; update IoT fields only if source.iotLastTs is newer; insert if new device
Upsert - iot_events_latest	MERGE on deviceID; update all fields if source.ts > target.ts; insert if new device
Cache clear	spark.catalog.clearCache() every 60 batches

5. Target Delta Tables

Table	Layer	Write Mode	Partition
deltalake.bronze.iot	Bronze	Append	year / month / day
deltalake.silver.iot_events_latest	Silver	MERGE (upsert)	None
deltalake.silver.iot_events_latest_valid_new	Silver	MERGE (upsert)	None

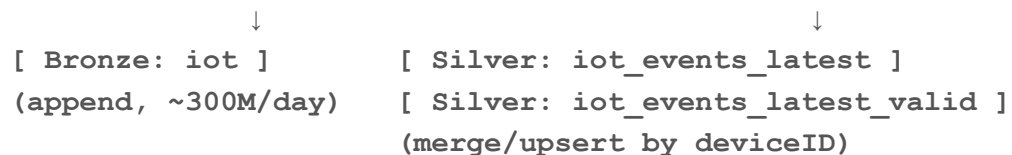
6. Alerting & Contacts

The current pipeline sends failure alerts to the email Ids we've passed in the yaml file

Webhook notification is also configured for Slack.

7. Appendix - Data Flow Diagram

[Kafka] → Avro decode → [Spark Structured Streaming]



8. [Link to the IOT Job on Databricks](#)